Chapter 9. Setting Safeguards

There's always a level of risk associated with GenAI applications. That's because they are built on top of foundational models, which are a nondeterministic technology that has the potential to provide inaccurate or hallucinated answers. Foundational models are also a general-purpose technology, so their responses may not always align with what you want them to do.

In this chapter, we discuss four patterns that can help you set safeguards around your GenAI applications. Template Generation (Pattern 29) is useful in situations where the risk involved in sending content without human review is very high but human review will not scale to the volume of communications. Assembled Reformat (Pattern 30) helps in situations where content needs to be presented in an appealing way but the risk posed by dynamically generated content is too high. Self-Check (Pattern 31) helps you identify potential hallucinations cost-effectively. Finally, Guardrails (Pattern 32) are a catchall way to apply safeguards around your core GenAI applications to ensure that they operate within ethical, legal, and functional parameters.

Pattern 29: Template Generation

8. Addressing Constraints
9. Setting Safeguards
Generative AI Design Patterns
10. Composable Agentic Workflows

time, all the application needs to do is deterministic string replacement on the reviewed template. This makes the final responses safe to send to consumers without additional review.

Problem

LLMs are a powerful technology, but they're not deterministic, so there's always some risk that their responses will be inaccurate or toxic.

For example, suppose you're a tour operator who's generating thank-you notes to people who purchased your tour packages. You want these thank-you notes to be personalized and highly readable, maybe even in multiple languages. You're very tempted to use an LLM to generate these notes, but you know that would mean exposing your brand to considerable risk. What if the notes contain inappropriate language or try to upsell inappropriate or controversial items? You could add a human-review step, but with potentially thousands of purchases a day, human review will get expensive.

Is there a way to use LLMs to generate the thank-you notes but avoid the expense of having to subject every note to human review?

Solution

Instead of using the LLM to generate the thank-you notes directly, you can use an LLM to generate templates for the thank-you notes (see Figure 9-1). The templates can be reviewed by humans and edited appropriately. You can also use few-shot learning and many of the patterns in Chapter 2 through Chapter 8 to ensure that the resulting content is grounded, in your brand voice, and so on, to keep the editing work minimal.

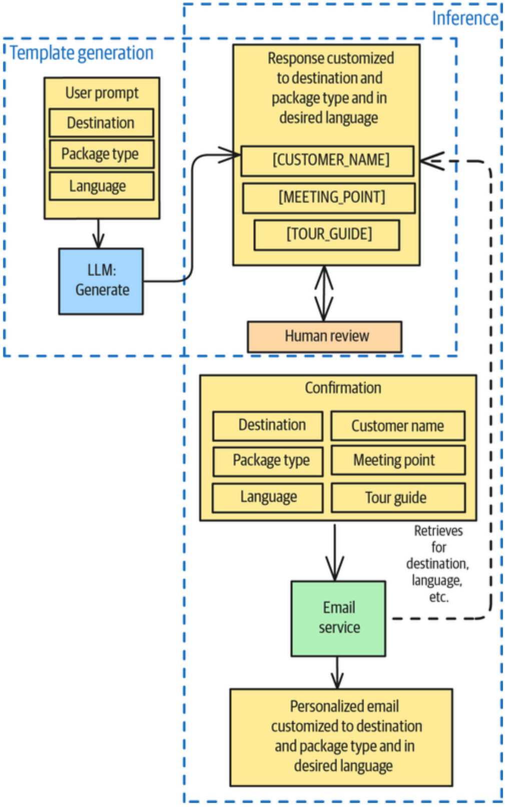


Figure 9-1. Template Generation: creating a few templates that can be reviewed by humans and then used as is during inference

At inference time, the application just needs to plug values into the template and send out the notes. Because this process is deterministic and constrained, the chances of introducing factual errors or toxic content at this stage are minimal.

Example

You operate a package tours company, and shortly after each booking, you send out a personalized thank-you note from the guide who will lead the tour because it tends to reduce the number of cancellations. The code for this example is on GitHub.

Pregeneration loop

You can pregenerate a templated thank-you note for every combination of tour, package type, and language that you offer and store these templates in a database for easy retrieval:

```
DESTINATIONS=[
    "Toledo, Spain",
    "Avila & Segovia",
    "Escorial Monastery"
]
PACKAGE_TYPES=[
    "Family",
    "Individual",
    "Group",
    "Singles"
]
LANGUAGES=[
    "English",
    "Polish"
]
for dest in DESTINATIONS:
   for package_type in PACKAGE_TYPES:
      for lang in LANGUAGES:
          template = create_template(dest, package_type, lang)
          db.insert(dest, package_type, lang, template)
```

Your business offers three destinations, four package types, and tours in two languages. So, you will generate 3 × 4 × 2 = 24 templates. To create each template, you can prompt a foundational model.

Generating a template

Some placeholders (such as language) will be replaced by the pregeneration loop, but the generated template will still contain placeholders for items like the customer name. These remaining placeholders will have to be filled in before sending:

```
def create_template(tour_destination, package_type, language):
    prompt=f"""
    You are a tour guide working on behalf of Tours GenAI S.L.
personalized letter in {language} to a customer who has purchas
{package_type} tour package to visit {tour_destination}. Sound
them and lead them on the tour. Explain some of the highlights
see there and some of the things they can do while there. In th
[CUSTOMER_NAME] to indicate the place to be replaced by their n
[TOUR_GUIDE] to indicate the place to be replaced by your name.
    """
    template = zero_shot(GEMINI, prompt)
    # ask humans to edit/confirm ...
    template = human_edit_confirm(template)
    return template
```

The generated template for an English-language tour of Toledo, Spain, might look like this:

> Dear [CUSTOMER_NAME],
>
> I'm absolutely thrilled to welcome you to Toledo! I'm [TOUR_GUIDE], and I'll be your guide for your family tour. I'm so excited to show you and your family this incredible city.
>
> ...
>
> Here's a sneak peek of what awaits you:
>
> * **The magnificent Toledo Cathedral:** A masterpiece of Gothic architecture that will leave you breathless.
>
> * **The Alcázar of Toledo:** A formidable fortress with panoramic views of the city.
>
> ...
>
> I can't wait to meet you and your family in person and share my passion for Toledo with you. Get ready for an unforgettable adventure!
>
> See you soon,
>
> [TOUR_GUIDE]

On the other hand, a Polish-language note for a tour of Avila and Segovia, Spain, might start with this:[1]

> Szanowni Państwo, [CUSTOMER_NAME]!
>
> Z ogromną radością witam Państwa w imieniu Tours GenAI S.L.! Jestem [TOUR_GUIDE] i będę miał przyjemność być Państwa przewodnikiem podczas rodzinnej wycieczki do Avili i Segowii!

Inference

Whenever a tour is purchased or a tour guide is confirmed for the tour, your application will invoke an email service with details of the tour. The application will retrieve the appropriate template from the database and replace the placeholders with strings from the session to obtain the body of the email:

```
booked_tour = ...
template = db.retrieve(booked_tour.destination,
                       booked_tour.package_type,
                       booked_tour.language)
email_body = template.replace(
               "[CUSTOMER_NAME]", booked_tour.customer_name
             ).replace(
               "[TOUR_GUIDE]", booked_tour.tour_guide.name
             )
# send out email
```

Considerations

Template Generation helps you avoid the expense and latency associated with conducting a human review of every piece of generated content. It works whenever the number of templates needed is tractable. If the number of combinations is too large, consider Assembled Reformat (Pattern 30 in this chapter). Another alternative to Template Generation is to use Guardrails (Pattern 32 in this chapter), but that adds considerable engineering complexity.

You can combine Template Generation with ML in a wide variety of personalization and customization scenarios—for example, you can build a set of personalized landing pages by using pregenerated templates and then use ML to select which templates to employ for a given marketing campaign or user profile. For personalized recommendations, you could use a traditional recommendations engine to select which products to show the user and pull in the appropriate pregenerated content (images and text) for that product.

You can also use this approach of combining pregeneration with ML if you're customizing your website for different customer journeys. In that case, you'd pregenerate the landing pages and use a *propensity model* (which is an ML model that predicts the likelihood that an event, such as a purchase, will happen) to choose the next best action.

References

Mail merge is a feature that personalizes documents or emails for mass distribution by combining a main template with data from a separate source. It dates back to a 1980s word processor called WordStar. The idea of creating these templates, rather than the final documents, with LLMs was introduced in 2024 by Valliappa Lakshmanan in an article on balancing creativity and risk in GenAI applications.

Pattern 30: Assembled Reformat

Assembled Reformat reduces the risk of inaccurate or hallucinated content by separating out the task of content creation into two low risk steps. The first step involves assembling raw data by using low-hallucination methods such as OCR, RAG, Tool Calling (Pattern 21), and Template Generation (Pattern 29). The second step involves reformatting the assembled content by using LLMs, since tasks like rephrasing and summarizing are relatively unlikely to introduce inaccuracies.

Problem

Suppose you're creating the product catalog for an ecommerce site. Product catalogs need to be appealing, both to potential buyers and to search engines. There are hundreds of thousands of product pages on the site, so you'd like to use LLMs to generate the web pages of the catalog.

What's the risk associated with hallucinated or inaccurate content in this context? For example, if the LLM-generated catalog page for a camera with a lithium battery says that the battery is alkaline, to what level of risk have you exposed your company? Lithium batteries aren't allowed in checked airline baggage because they can cause uncontrollable fires in enclosed spaces. What if a camera from your site ignites or accelerates a fire in the cargo hold of an airplane? What if an airline won't let your customer board an airplane because they have a flammable item in their baggage? We hope you'll agree that the risk associated with this seemingly simple error in battery type seems rather high, so dynamic generation is too risky for this use case.

Is there a way to get the benefits of LLM generation without incurring the risks posed by LLMs' potential to introduce inaccurate or hallucinated information?

Solution

The Assembled Reformat pattern works if you can identify the characteristics of the product that would incur unacceptable risk if hallucinated, and you can assemble these characteristics by using low-risk methods, such as reading from a database or using a document data extraction system.

Creating the final result by either appending all the text that corresponds to the product attributes or putting them into some predefined structure results in text that's accurate but not very pleasing to read.

Once you have this accurate text, put it into the context of the prompt and ask the LLM to rephrase, reword, or reformat it (see Figure 9-2). Text generated by these LLM tasks tends to be much lower risk than text generated from scratch, and the resulting text will be fluent and better suited to the content's purpose.

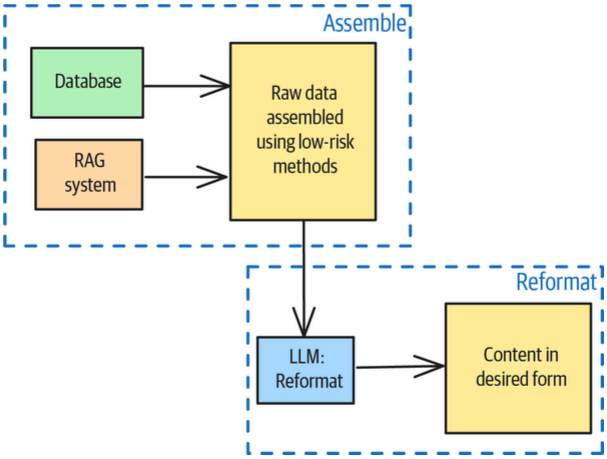


Figure 9-2. Assembled Reformat reduces the risk associated with content creation by separating the task into two low-risk steps

Example

For the product catalog case (the full code is on GitHub), you could define the raw data to be collected as a data class:

```
@dataclass
class CatalogContent:
    part_name: str = Field("Common name of part")
    part_id: str = Field("unique part id in catalog")
    part_description: str = Field("""One paragraph description
one of the parts of a paper machine, explaining its purpose."""
    failure_modes: list[str] = Field("""list of common reasons
need to replace this part.""")
    warranty_period: int = Field("""number of years that the pa
warranty""")
    price: str = Field("price of part")
```

It might retrieve the part name, ID, warranty period, and price from a database but pull the part description and failure modes (see italicized text above) from an equipment manual. If you're using an LLM for this kind of data extraction, use a low temperature, perhaps 0.1 or 0, to lower the risk of inaccuracy.

For a part called a *wet end* that's used in paper-making machines, the catalog content might be something like this:

```
CatalogContent
part_name='wet_end'
part_id='X34521PL'
part_description='The wet end of a paper machine is the section
web is formed. It is arguably the most important section of the
failure_modes=['Web breaks', 'Uneven sheet formation', 'Poor dr
warranty_period=3
price='$23295
```

The failure modes extracted from trusted sources have been italicized. By grounding subsequent generation on these three failure modes, the LLM will always include caveats on these three issues and will also not include other failure modes that the paper machine manufacturer might not want publicized.

Once you have the extracted content, you can pass it along to a more creative writing engine that will reformat it to meet your desired goal, such as search engine optimization and/or being persuasive to potential buyers:

> Write content in Markdown that will go in the Replacement Parts part of the manufacturer's website. Include a placeholder for an image and include a description of the image. Optimize the content for SEO. Also make it appealing to potential buyers.
>
> **Part Information:**

> {item}

The resulting text uses the information about the three "acceptable" failure modes in a way that justifies why you might want to buy the part:

> The wet end is where the magic happens—the initial formation of the paper web. A poorly functioning wet end can lead to a cascade of problems, including:
>
> * **Web Breaks:** Frequent web breaks are a major source of downtime and waste of materials.
>
> * **Uneven Sheet Formation:** Inconsistent sheet formation results in lower-quality paper and potential customer complaints.
>
> * **Poor Drainage:** Inefficient drainage reduces machine speed and increases energy consumption.
>
> Investing in genuine replacement parts for your wet end is an investment in the overall efficiency and quality of your paper production.

Considerations

Even though the two steps (assembly and reformatting) are low risk, you should validate the results. For example, if possible, extract the data in two ways to see if you get the same result. You can also use Pattern 31, Self-Check (which we cover next), for this purpose. Similarly, even though reformatting is low-risk, you should validate that the generated content retains the raw data that should not be hallucinated. You can use Pattern 17, LLM-as-Judge, for this.

If you're considering using Assembled Reformat, first consider whether Pattern 29, Template Generation, will suit your needs—its ability to review all templates provides an extra safeguard. Choose Assembled Reformat only in situations where you can't use Template Generation, perhaps because you have more items than would be possible for humans to review.

The Assembled Reformat approach works for web pages where the content is somewhat static (as in product catalog pages). However, if you want to customize your pages to the user's journey or profile (as in marketing landing pages), the content will need to be much more dynamic. For that, you'll need Template Generation.

References

Assembled Reformat was introduced in 2024 by Valliappa Lakshmanan in an article on balancing creativity and risk in GenAI applications.

Pattern 31: Self-Check

The Self-Check pattern uses token probabilities to detect hallucination in LLM responses. You can use this as a safeguard against the LLM providing low-confidence answers to factual queries.

Problem

When the LLM generates incorrect, nonsensical, or fabricated content that isn't consistent with the real world or the input context, we call that generated response a *hallucination*. Hallucinations happen because LLMs are, at their core, statistical token generators. They don't know the meaning of what they generate, and in the absence of actual knowledge, the responses they generate might not be factually correct.

As LLM providers have improved the coverage of their training data and methodologies, hallucination rates for common tasks and queries have dropped steadily. For example, Vectara measured the top 25 LLMs' hallucination rates on a text summarization task, as shown in Figure 9-3. In December 2024, the best LLM was hallucinating at a rate of 1.3% and the 25th best was hallucinating at a rate of 4.1%. When Vectara tested the same measure on the same task in April 2025, hallucination rates had dropped by 40% to 50% across the board—the best LLM's hallucination rate was now 0.7% and that of the 25th was 2.4%.

Hallucination Rate for Top 25 LLMs

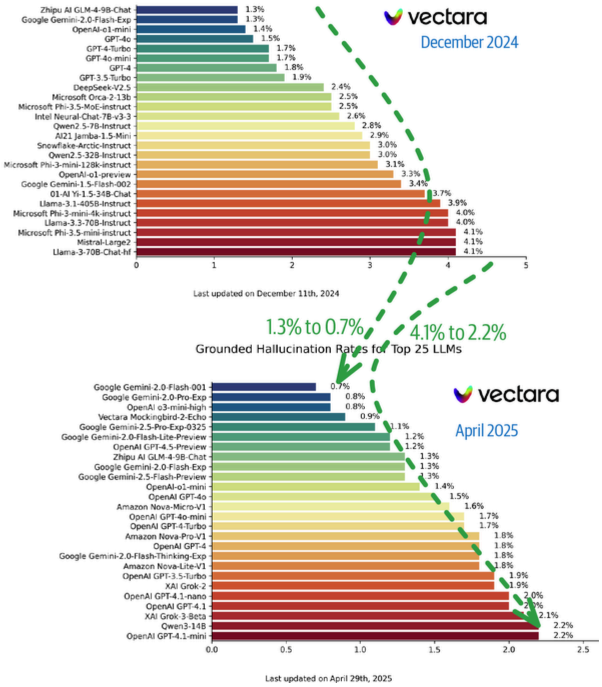


Figure 9-3. Hallucination rates are dropping (images from Vectara)

However, hallucination remains a problem in more constrained or complex situations. For example, say you're using an LLM to extract numbers from images but one of the images is smudged. If you constrain the response to be a number (see Pattern 2, Grammar), you'll get back a hallucinated number.

Cases of smudging are easy to understand, but the problem remains even with pristine input. At the time of writing, the accuracy rate in fields extracted from images is on the order of 90% to 97%. This means that 3% to 10% of the numbers extracted from invoices, receipts, and the like are hallucinated! Moreover, if you now take the output of the data extraction and pass it as the input context of other LLM calls, the chances of errors occurring compound—even if you have error detection and correction in place. The more complex your LLM-calling chain, the more likely it is that the LLM responses will be hallucinated.

Is there a way to determine whether an LLM is hallucinating? Suppose you have three LLMs that were trained in very different ways on nonoverlapping datasets. You could use these three LLMs to generate responses and compare their responses. When the responses differ, there is a high likelihood of hallucination—and while it might be unclear what the correct answer is, at least you'd know that there's a potential problem. Intuitively, then, you can use the variability of LLM responses to identify potential hallucination trouble spots.

However, the frontier models' training datasets overlap quite a lot, so it would be difficult to source three such nonoverlapping LLMs. Also, inference with multiple LLMs will multiply the costs. Is there a way to look at an LLM response and identify potential hallucination trouble spots?

Solution

As we discussed in Chapters 1 and 2, many LLMs provide, along with the generated tokens, those tokens' *logprobs* (also called *logits*). From the logprobs, you can calculate the probability of a specific token being the correct one as follows:

e^{logit}

In situations where the generated token is the overwhelming favorite, this probability will be close to 100%. In situations where there are multiple possibilities (see Figure 1-5), the probability of the "winning" token being the correct one will be lower.

Requesting and processing logprobs

You can ask OpenAI to return the logprobs of each token along with the response text. The following code also asks for the five leading candidates at each step (this code is on GitHub):

```
message = client.chat.completions.create(
```

```
        model="gpt-3.5-turbo",
        messages=[
            ...
        ],
        logprobs=True,
        top_logprobs=5
    )
```

You can then retrieve and process the three requested components of the response message as follows:

```
response_text = message.choices[0].message.content
logprobs = message.choices[0].logprobs
for token_info in logprobs.content:
    token = token_info.token
    logprob = token_info.logprob
    probability = math.e ** logprob
    if token_info.top_logprobs:
        for alt_token in token_info.top_logprobs:
            if alt_token.token != token:
                alt_probability = math.e ** alt_token.logprob
```

How logprobs behave

Suppose you ask GPT-3.5 about the founder of the Republic of Turkey:

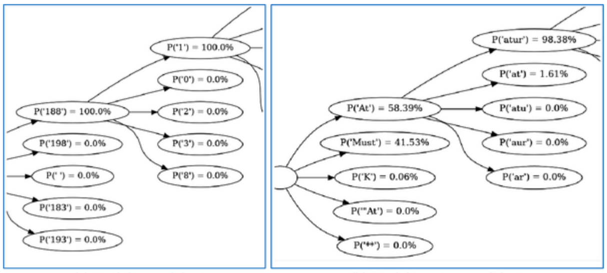
> What year was Ataturk born? Answer in one sentence.

The model responds with this:

> Ataturk was born in 1881.

The year 1881 is represented as two tokens, *188* and *1*. Their probabilities are shown in Figure 9-4a. As you can see, the model is quite confident in this answer. The other candidates, such as him being born in the 1980s, 1830s, or 1930s, have probabilities that are all near zero.

This doesn't mean that all low probabilities are suspect. The probabilities of the candidate tokens at the start of the sentence are shown in Figure 9-4b.



(a) Model is confident

(b) Model is not so confident

Figure 9-4. (a) Probability of tokens corresponding to generating the year part of the answer—(b) Probability of tokens corresponding to the start of the answer, where there are several possibilities

Why is the probability of the *At* token only 58%? The next candidate is *Must*, and that gives us a clue. The following is also a correct answer to the question:

> Mustafa Kemal Atatürk was born in 1881.

The presence of the umlaut over the *u* in the last name also explains why the second possible token at the second position is *at*—it leaves space for an umlaut to appear as the third token of the name.[2]

We hope this has given you insight into how the logprobs behave when the model is confident and when there are many alternative continuations.

Low-confidence answers

Now, let's take a look at a situation in which the model hallucinates. We'll purposely use an older model here in the hope that it will not have been fixed to handle this hallucination error.

We ask GPT-3.5-turbo the following question:

> Who is John Cole Howard? Answer in one sentence.

The model responds with this:

> John Cole Howard is a fictional character from the TV show ***The*** *Office*, portrayed by actor **Ed** Helms.

The tokens that were selected even though they didn't reach a 50% probability were *The* and *Ed* (which are bolded in the preceding response and shown in Figure 9-5).

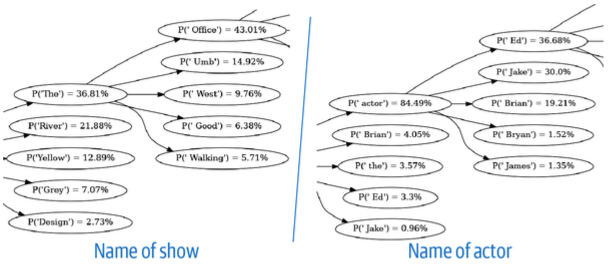


Figure 9-5. Logprobs at low-confidence tokens

This is likely because Ed Helms' character in *The Office* is Andy Bernard and there is no famous person named John Cole Howard. Hence the low probability on the name of the show and on the name of the actor: the model is pretty much guessing at this point.

Identifying hallucinations

Given that the model emits tokens with low values of logprobs when it has to choose from many equally likely options, you could threshold the logprobs and use low probabilities as indicators of possible hallucinations. However, you have to be careful about false positives. As the Atatürk example indicates, there are valid reasons for a correct answer to include tokens that have low probabilities.

There are several approaches that you can use to limit the false positives (also see Figure 9-6):

Identify tokens of interest

To limit false positives, you can focus on checking the logprobs only on specific tokens. This is often possible when you're generating structured output because it's possible to determine the positions of the key values to validate and check the logprobs of only those tokens. We illustrate this in the following "Example" section.

Sample generated sequences

Another way to limit false positives is to use the sequence generation approach whenever you encounter low-probability tokens (see "Pattern 1: Logits Masking" in Chapter 2 for a detailed walkthrough of sequence generation). The idea is that you can generate multiple sequences and validate that they all agree on the answer.

In the case of the Atatürk example, even though the sentences may start differently, both generations would have agreed on his birth year being 1881. You can compare whether the answers are substantially the same by comparing the embeddings of the two generations.

Normalize statistics over all tokens

Calculating aggregate statistics over long answers can underestimate (in the case of averages) or overestimate (in the case of minimums) the hallucination potential. An aggregate statistic that normalizes the logits for sequences of different lengths is *perplexity*, which is defined as follows:

$$perplexity = e^{-\frac{1}{N}\Sigma_i^N logits_i}$$

So, the perplexity is the number of alternatives between which the model is choosing. The lower the perplexity, the more confident the model is in the generated sequence.

Build an ML model

You can treat the token probabilities of specific probabilities, the distance between embeddings of generated sequences, aggregate and normalized statistics, and contextual features as input features into an ML model that's trained on your data and specific use case to detect hallucination.

Using a bespoke ML model is the most robust approach because it builds on all of these methods.

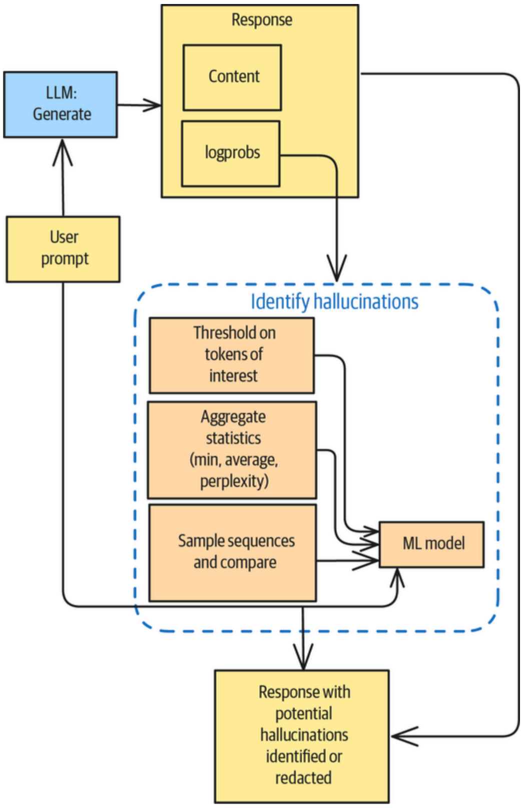

Response
Content
logprobs
LLM: Generate
User prompt
Identify hallucinations
Threshold on tokens of interest
Aggregate statistics (min, average, perplexity)
Sample sequences and compare
ML model
Response with potential hallucinations identified or redacted

Figure 9-6. Detecting hallucinations with Self-Check

Example

Suppose you're writing software to process signed receipts at a restaurant. The restaurant is in a country where tipping is common and restaurant meals are taxed, so you'll need to extract four numbers from each receipt: the billed amount, the tax, the tip, and the total amount.

Suppose your extraction code returns the following four values:

> 312.32,28.76,60,401.08

If you extract all four numbers, the total amount acts as a *checksum*—you can calculate it from the other three numbers to confirm that the extracted value is correct.

But suppose you can extract only three of the numbers (perhaps the fourth image is smudged). You get this back:

> 312.32,28.76,,400

You want the LLM to impute the number that could not be extracted, and you can do that with a prompt (the full code is on GitHub):[3]

> You are a helpful AI assistant that helps parse restaurant receipts.
>
> I will give you a set of parsed values containing the following on each line:
>
> billed_amount, tax, tip, paid_amount
>
> If tax is missing, calculate it as 9.21% of the billed_amount.
>
> If the tip is missing, calculate it as (paid_amount - billed_amount - tax).
>
> If the paid_amount is missing, calculate it as (billed_amount + tax + tip).
>
> Do not add any headers or explanations.

You can parse the LLM result as follows:

```
parse_result(response_text=message.choices[0].message.content,
```

```
             logprobs=message.choices[0].logprobs)
```

The content can be parsed into a Pandas DataFrame:

```
def parse_result(response_text, logprobs) -> pd.DataFrame:
   csv_file = StringIO(response_text)
   result_df = pd.read_csv(csv_file, header=None,
                  names=['billed_amount', 'tax', 'tip', 'paid_a
```

Along with the content, the model sends back logprobs. You can associate each returned token with the line it appears in and compute the lowest-confidence token based on that line:

```
line_no = 0
confidence_of_line = 1.0
last_col_no = len(result_df.iloc[0]) - 1
for token_info in logprobs.content:
    token = token_info.token
    logprob = token_info.logprob
    probability = (2.718281828459045 ** logprob)
    confidence_of_line = min(confidence_of_line, probability)
    result_df.iloc[line_no, last_col_no] = confidence_of_line
    if '\n' in token: # next line
       line_no = line_no + 1
       confidence_of_line = 1.0
```

Let's try this out. Suppose you send the following data to the LLM (OpenAI's GPT-4o-mini, in this case):

312.32,28.76,60,401.08

312.32,28.76,,400

312.32,28.76,60,

312.21,,50,

312.43,,,400

300,27.63,60,387.63

You might get back the Pandas DataFrame shown in Table 9-1:

Table 9-1. Pandas DataFrame generated by GPT-4o-mini

billed_amount	Tax	Tip	paid_amount	Confidence
312.32	28.76	60.0	401.08	0.962668
312.32	28.76	60.0	400.00	0.551552
312.32	28.76	60.0	400.08	0.562172
312.21	28.84	50.0	391.05	0.172516
312.43	28.80	60.0	401.23	0.170295
300.00	27.63	60.0	387.63	0.999290

Note how, in the last column, the confidence is high *only* for the two rows where nothing was imputed. The confidence is moderate (around 0.55) for the two rows where only one value had to be imputed, and it's low (0.17) for the columns where two values were imputed.

You can identify, solely by looking at the confidence value, which rows of the parsed table are problematic. Indeed, if you calculate the checksum error in the `paid_amount`, the rows with no error have a confidence above 0.9. In a complex chain where thousands of tokens are being generated in a constrained way, you can use the generating LLM's own confidence scores to identify potentially problematic outputs.

Considerations

A simpler method than Self-Check, and quite an effective one in many situations, is to explicitly provide the model an out. For example, you can ask the model to respond, "I don't know," when asked a question that is outside its training data. When generating structured outputs (see Pattern 2, Grammar), you can model a field as a union where one of the alternatives is that the model doesn't know:

```
currency_rate: float | Literal["Unknown"]
```

Self-Check can be extremely helpful in identifying inconsistent data in RAG. If two retrieved chunks contradict each other, the generated response will have logprobs that indicate that there were two possible generation paths. Regardless of which one was selected, you can look at the logprobs to identify potentially problematic answers. Of course, the fact there are two possible paths doesn't mean that there is a conflict—both paths could lead to the same answer. You can use the more robust approaches that are detailed in the "Solution" section to limit the number of false positives.

As discussed in the "Caveats" subsection of "Pattern 1: Logits Masking" in Chapter 2, not all models provide access to their logprobs.

References

Manakul, Liusie, and Gales (2023) introduced the idea of using logprobs for hallucination detection. They used sequence generation, called their detector SelfCheckGPT, and suggested using a separate LLM solely to generate logprobs when the proprietary LLM doesn't itself provide logprobs. Quevedo et al. (2024) trained an ML classifier on the token probabilities output by the LLM to detect hallucinations. Valentin et al. (2024) compared various ways of using logprobs to detect hallucinations.

Pattern 32: Guardrails

Guardrails are layers of code that operate on an LLM's inputs, outputs, context, and tool parameters to safeguard the LLM from malicious actors and ensure that it operates within allowed parameters.

Problem

When building AI applications, you'll often need to ensure that they operate within specific ethical, legal, and functional parameters. For example, you may need to safeguard your AI applications in the following areas:

Security

> Protecting your AI systems from malicious inputs typically requires you to guard against prompt injection attacks and jailbreaking. Prompt injections exploit the fact that LLMs process both system prompts and user inputs as text, which makes it difficult for models to distinguish between legitimate instructions and malicious commands. Prompt injections may be direct, with attackers feeding malicious prompts to the LLM, or indirect, with attackers hiding payloads in data the LLM consumes. For example, Carnegie Mellon researchers found in 2023 that suffixes that appear to be random characters can cause LLMs to behave in unexpected ways.

Data privacy

> You need to guard against your AI systems inadvertently exposing sensitive information, such as personally identifiable information (PII), trade secrets, or confidential content. This could happen if such sensitive information was present in the AI systems' training data or in cached versions of user inputs. Exposing sensitive data could lead to privacy breaches and potential legal issues.

Content moderation

> You'll often need to filter harmful, toxic, or inappropriate content from both user inputs and model outputs. This may be less necessary in internal-facing applications than in public-facing ones. LLMs can generate or respond to content that includes hate speech, violence, sexual material, or other harmful content, potentially causing harm to users or damaging brand reputation.

Hallucination

> You may need to ensure that LLM outputs are accurate, truthful, and grounded in reliable information. LLMs can generate plausible-sounding but factually incorrect information. This can be particularly problematic in applications where accuracy is critical, such as science, journalism, health

care, law, and finance.

Alignment

You may need to ensure that LLM outputs adhere to specific guidelines, company policies, or ethical principles. For instance, your organization may require that all company communications adhere to its specific policies, guidelines, and brand voice, or that outputs avoid mentioning competitors or topics such as politics or religion. You may also operate in a jurisdiction or industry that imposes ethical boundaries to prevent bias and discrimination and ensure fairness among different demographic groups.

While these factors are important, you don't want to sprinkle your application code with security, privacy, and content checks. A large error-handling surface area will be hard to maintain and enforce. Can you safeguard your AI applications in all these areas while keeping the maintenance overhead low?

Solution

With the Guardrails pattern, you can implement a layer of guardrails to provide comprehensive protection at different points in the conversation flow between the foundational model and inputs, outputs, knowledge bases, and tools (see Figure 9-7).

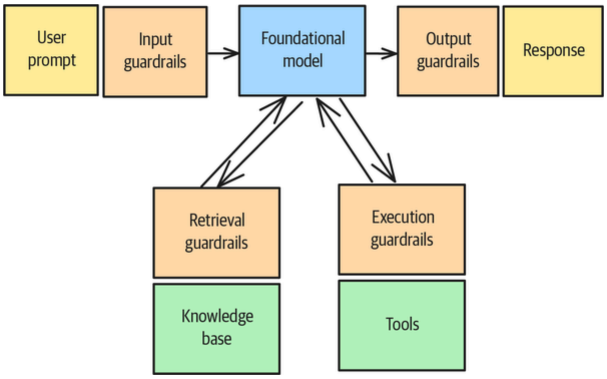


Figure 9-7. Guardrails are layers that are inserted to safeguard all inputs and outputs going into and out of the LLM

Guardrails involve preprocessing the information that's going into the model and/or post-processing the output of the model. That processing might involve modifying the input or output to correct for errors or outright rejecting it.

Prebuilt guardrails

Some LLMs have built-in safety features that you can turn on when invoking the model through its API. For example, on Gemini, you can block hate speech from being generated by using this code:

```
response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents=[prompt, media, ...],
    config=types.GenerateContentConfig(
      safety_settings=[
        types.SafetySetting(
            category=types.HarmCategory.HARM_CATEGORY_HATE_SPEE
            threshold=types.HarmBlockThreshold.BLOCK_LOW_AND_AB
        ),
      ]
    )
)
```

Frameworks such as NVIDIA's NeMo, Guardrails AI, and LLM Guard provide prebuilt guardrails for common functionalities such as checking for jailbreaks, masking sensitive data in the input, and hallucinations.

For example, here's how to use LLM Guard to scan a prompt for toxic language:

```
from llm_guard.input_scanners import Toxicity

scanner = Toxicity(threshold=0.5, match_type=MatchType.SENTENCE
sanitized_prompt, is_valid, _ = scanner.scan(prompt)
```

The scanner takes a string input and returns a sanitized version of the input (which is useful for guardrails that redact PII, for example) and decides whether the input should be allowed.

Protecting against prompt injection is very similar:

```
scanner = PromptInjection(threshold=0.5, match_type=MatchType.F
sanitized_prompt, is_valid, _ = scanner.scan(prompt)
```

In both cases, the framework uses post-trained small language models (SLMs) to carry out the task. For toxicity, it uses the unitary/unbiased-toxic-roberta library, and for prompt injection, it uses the ProtectAI/deberta-v3-base-prompt-injection-v2 library.

However, LLM Guard also supports rejecting strings that match a regular expression:

```
scanner = Regex(
    patterns=[r"Bearer [A-Za-z0-9-._~+/]+"],  # List of regex p
    is_blocked=True,  # If True, patterns are treated as 'bad'
    match_type=MatchType.SEARCH,  # Can be SEARCH or FULL_MATCH
    redact=True,  # Enable or disable redaction
)
sanitized_prompt, is_valid, risk_score = scanner.scan(prompt)
```

Custom guardrails

You can build custom guardrails by implementing the logic in your code, prompting foundational models, or post-training an SLM. Here's a guardrail that illustrates how to use Pattern 17, LLM-as-Judge, to reject prompts on specific topics:

```
banned_topics = [
        "religion", "politics", "sexual innuendo"
]
system_prompt=f"""
I will give you a piece of text. Check whether the text touches
topics.

        {banned_topics}

Return True or False, with no preamble or special markers.
Text:
"""
llm = ...
response = llm.complete(prompt).text.strip()
is_valid = (response == "False")
```

Applying a set of guardrails

Once you have a set of guardrails, apply them one after the other:

```
def apply_guardrails(guardrails, prompt):
sanitized_prompt = prompt # initial
for scanner in guardrails:
   sanitized_prompt, is_valid, _ = scanner(sanitized_prompt)
   if not is_valid:
      raise Exception("...")
return sanitized_prompt
```

Applying guardrails to the output, retrieval, or tool parameters works similarly.

Example

Let's take an example of a RAG system that's designed to answer questions based on Jane Austen's writings. (The full code is on GitHub.)

Say you ask the RAG system a question such as the following:

> Can you give advice without being resented for it?

You'll get back a response similar to this:

> Yes, it is possible to give advice without being resented for it. The text shows an example ... that is described as "a wonderful instance of advice

> being given on such a point, without being resented.”
>
> However, the manner in which advice is offered seems important. In another example, ...

Implementing guardrails

Suppose you want the system to prevent users from sending you PII. You want any proper names in prompts to be replaced by generic identifiers, and you can do that with a custom guardrail:

```
def guardrail_replace_names(to_scan: str):
    llm = ...
    system_prompt="""
I will give you a piece of text. In that piece of text, replace
names with a generic identifier.

Example:
          Input:
            I met Sally in the store.
          Output:
            I met a woman in the store.

Return only the modified text, with no preamble or special mark
    """
    sanitized_output = llm.complete(system_prompt + "\n" + to_s
    no_change = (sanitized_output == to_scan)

    return {
        "guardrail_type": "PII Removal",
        "activated": not no_change,
        "should_stop": False,
        "sanitized_output": sanitized_output,
    }
```

Similarly, you can reject any prompts that touch on a set of banned topics by using the following code. (This uses an LLM to implement the guardrail.):

```
def guardrail_banned_topics(to_scan: str):
    banned_topics = [
        "religion", "politics", "sexual innuendo"
    ]
    llm = ...
    system_prompt=f"""
I will give you a piece of text. Check whether the text touches
topics.

        {banned_topics}

Return True or False, with no preamble or special markers.
Text:
    """
    response = llm.complete(system_prompt + "\n" + to_scan).tex
    is_banned = (response == "True")

    return {
        "guardrail_type": "Banned Topic",
        "activated": is_banned,
        "should_stop": is_banned,
        "sanitized_output": to_scan,
    }
```

To make it easier to apply multiple guardrails, all of your guardrail functions should have the same signature.

Wrapping the query engine

You can wrap the query engine provided by LlamaIndex with a set of guardrails:

```
class GuardedQueryEngine(RetrieverQueryEngine):
    def __init__(self, query_engine: RetrieverQueryEngine):
        self._query_engine = query_engine

    def query(self, query):
        # apply guardrails to inputs
        gd = apply_guardrails(query,
                 [guardrail_replace_names, guardrail_banned_top
        if not gd["should_stop"]:
            print(f"Modified Query: {gd['sanitized']}")
```

```
            query_response = self._query_engine.query(gd["sanit
            gd = apply_guardrails(str(query_response),
                                  [guardrail_banned_topics])
            if not gd["should_stop"]:
                return Response(gd["sanitized"],
                               source_nodes=query_response.sour
        return Response(str(gd))
```

The wrapped version applies the set of guardrails to the inputs and then passes the sanitized prompt to the original query engine. It then applies a different set of guardrails to the response and returns it only if neither the input nor the output is blocked.

Because religion is one of the banned topics, the RAG system refuses to answer the following question:

> Are parish priests expected to be role models?

Since proper names are to be redacted, the following query must be modified before it is sent to the LLM:

> Would Mr. Darcy be an appealing match if he were not wealthy?

The modified version would be as follows:

> Would a man be an appealing match if he were not wealthy?

Considerations

Guardrails introduce considerable engineering complexity and latency into your application architecture. The most complex way in which you could choose to implement a GenAI application is by deploying it alongside custom post-trained guardrails, so make sure that this complexity is warranted. Perhaps there are less expensive ways you can safeguard against the key risks you're concerned about.

Even though the guardrails in the example were implemented by calling out to a frontier model API, this need not be the case: it's possible to use SLMs to keep latency within manageable limits.

It is not necessary to run the guardrails and LLM code sequentially—for example, you could run input and/or retrieval guardrails in parallel with the incoming request to avoid slowing down the user's request:

```
try:
input_guardrail_results, turn_result = await asyncio.gather(
    apply_guardrails(
        ...
    ),
    llm.complete(
        ...
    ),
)
except InputGuardrailTriggered:
...
```

If the input guardrail detects disallowed usage, it should raise an error, and this will stop the second call from continuing to execute. Of course, if you do this, the LLM will not be protected from executing the call—instead, you'll be focused only on protecting your application from using the results of a malicious input.

There are inherent tradeoffs among security, usability, and performance. Stricter guardrails may reduce model capabilities or increase latency, and attackers can bypass even the most sophisticated guardrails with sufficient effort. Because attack techniques also evolve, you'll have to think of guardrails as an ongoing arms race between security measures and bypass methods. Given this reality, it's worth thinking of your guardrails as a wrapper around your AI applications that you change every few months. Instead of exerting effort building highly customized guardrails, build ones that you can easily port over to a new framework or ones that are model agnostic.

The strategy is to keep updating an evaluation dataset that consists of situations that you want to guard against and the maximum latency you're willing to tolerate. Then, periodically test the commercially available guardrail systems and change your underlying framework and models as necessary.

References

Dong et al. (2024) explain why point solutions for guardrails don't work and a comprehensive approach is needed. The OWASP security project classified prompt injection scenarios.

QED42 built prompt-based guardrails with policies, exceptions, and few-shot examples around an LLM-powered search application at a legal entity. The guardrails filtered out out-of-domain (nonlegal) query inputs and ensured that the outputs were relevant. Acrolinx uses AI guardrails that are implemented with LLM-as-Judge to maintain brand voice consistency across content.

Summary

In this chapter, we explored four patterns for implementing safety mechanisms in AI applications, and we addressed critical concerns regarding security, data privacy, content moderation, hallucination prevention, and ethical alignment. Table 9-2 summarizes these patterns.

Table 9-2. Patterns for teaching capability

Patterns	Problems	Solutions	Usage scenarios
Template Generation (Pattern 29)	The risk of sending content without human review is very high, but human review will not scale to the volume of communications.	Pregenerate templates that are reviewed beforehand. Inference time requires only deterministic string replacement, and it's therefore safe to directly send to consumers.	Personalized communications in business to consumer settings
Assembled Reformat (Pattern 30)	Content needs to be presented in an appealing way, but the risk posed by dynamically generated content is too high.	Reduce the risk of generating inaccurate or hallucinated content by separating the task of content creation into two low-risk steps—first, assembling data in low-risk ways and second, formatting the content based on that data.	Situations where accurate content needs to be presented in appealing ways, such as in product catalogs
Self-Check (Pattern 31)	You need to identify potential hallucinations cost-effectively.	Use token probabilities to detect hallucination in LLM responses.	Any situation where factual (as opposed to creative) responses are needed
Guardrails (Pattern 32)	You need safeguards for security, data privacy, content moderation, hallucination, and alignment to ensure that AI applications operate within ethical, legal, and functional parameters.	Wrap the LLM calls with a layer of code that preprocesses the information going into the model and/or post-processes the output of the model. Knowledge retrieval and tool use will also need to be protected.	Anytime your application could be subject to attacks by malicious adversaries

---

[1] We asked a native Polish speaker to proofread this, and she said that the template was quite good. The template uses *Szanowni Państwo*, the plural form that is quite formal and quite appropriate for a more expensive tour package. However, she said she'd modify the template in two ways. First, Polish has grammatical gender, so depending on what's in place of the [TOUR_GUIDE], the verb *mieć* (to have) would have to be either in masculine form *miał* or feminine form *miała*. Second, there are too many exclamation points for a Polish audience. This demonstrates the need for the pattern—it's easier to fix a single template than to review thousands of generated letters.

[2] A reasonable inference from the token probabilities is that 98.4% of the documents on the Turkish statesman on which OpenAI trained its model omitted the umlaut in his name. This sort of training-data leakage may be why some proprietary models refuse to provide logprobs. If only one document ever spells his name with an umlaut, it would be proof that the document in question was used in training the model, even if the generated responses never contain an umlaut.

[3] This is a somewhat contrived example. You wouldn't use an LLM to do mathematical calculations. Instead, you'd arm the LLM with a calculator tool (see Tool Calling [Pattern 21] in Chapter 7). However, the same sort of effect would happen with other types of generation, except that the nondeterminism of LLM responses would make it much harder to illustrate for the purposes of this book. Using this contrived example also allows us to evaluate the correctness of the LLM result automatically.

Table of contents collapsed